

PRD — ATS CV Reviewer

Versi: 1.0 (MVP — Review Mode)

Stack: Backend Java Spring Boot, Frontend Next.js

1. Overview

Web tool yang menganalisa CV (PDF/DOCX) terhadap sebuah Job Description (JD) spesifik, meniru cara kerja ATS (Applicant Tracking System) asli yang dipakai HRD/recruiter, lalu memberikan skor, detail issue, dan saran rewrite per bullet. Tujuan: bantu user tahu apakah CV-nya bakal lolos gate otomatis sebelum sampai ke recruiter manusia.

2. Problem & Goal

Problem: Banyak kandidat gagal di tahap awal seleksi bukan karena gak kompeten, tapi karena CV-nya gagal di-parsing atau gak match keyword dengan JD — bukan karena skill kurang.

Goal produk:

- User bisa cek kesehatan CV-nya terhadap JD target dalam hitungan detik
- Feedback actionable (bukan cuma skor angka kosong)
- Hasil bisa didownload sebagai PDF report

Non-goal (MVP ini): Auto-generate CV baru yang udah direwrite penuh (masuk roadmap fase 2).

3. Target User

Job seeker yang mau apply ke perusahaan yang pakai ATS (Workday, Greenhouse, Taleo, iCIMS, dll), khususnya yang ngelamar ke banyak posisi dan mau optimasi CV per JD.

4. Tech Stack

Layer	Teknologi
Frontend	Next.js
Backend	Java Spring Boot
AI	LLM API (Claude/OpenAI, model murah seperti Haiku untuk hemat biaya)
PDF parsing	Apache PDFBox
DOCX parsing	Apache POI
PDF generation (report)	iText / OpenPDF
Rate limiting	Bucket4j
Auth (opsional)	Spring Security + JWT

5. Scope — MVP (Review Mode)

Output MVP: **skor + feedback + saran rewrite teks**. User copy-paste sendiri hasil rewrite ke CV mereka. **Bukan** auto-generate file CV baru.

6. User Flow

1. User buka web (tanpa wajib login)
2. Upload CV (PDF/DOCX) + paste JD target
3. Sistem proses (parsing → keyword matching → scoring)
4. User lihat hasil: skor per kategori + detail issue + saran rewrite per bullet
5. User bisa download PDF report
6. (Opsional) User login untuk simpan history analisa sebelumnya

7. Fitur Detail — 3 Gate (meniru pipeline ATS asli)

ATS asli memproses resume lewat 3 fase berurutan: **parsing → matching → ranking**. Kalau gagal di parsing, resume gak pernah sampai ke tahap matching. Sistem ini meniru urutan yang sama.

Gate 1 — Parsing Check (deterministic, bukan AI)

Dicek pakai code/heuristic, bukan LLM, supaya hasilnya konsisten tiap kali:

- Layout: deteksi multi-column, tabel, text box (parser gagal baca ini)

- Section heading: cocokkan ke whitelist ("Work Experience", "Education", "Skills", dll)
- Font: cocokkan ke whitelist font ATS-safe (Arial, Calibri, Times New Roman, Georgia)
- Posisi contact info: flag kalau ada di header/footer
- Tipe file: harus PDF/DOCX asli (bukan hasil convert dari gambar/.pages)

Gate 2 — Keyword Matching (butuh AI, semantic)

- Extract keyword wajib dari JD (skill, tools, sertifikasi, job title)
- Compare ke isi CV — matching cenderung literal (mis. "SEO" vs "Search Engine Optimization" dianggap beda kalau cuma salah satu yang ada)
- Cek posisi keyword: keyword yang nempel di bullet pencapaian dinilai lebih kredibel dibanding cuma disebut di skill list
- Cek keyword density: ideal 2-3x repetisi per keyword penting; di atas 4-5x kena flag stuffing

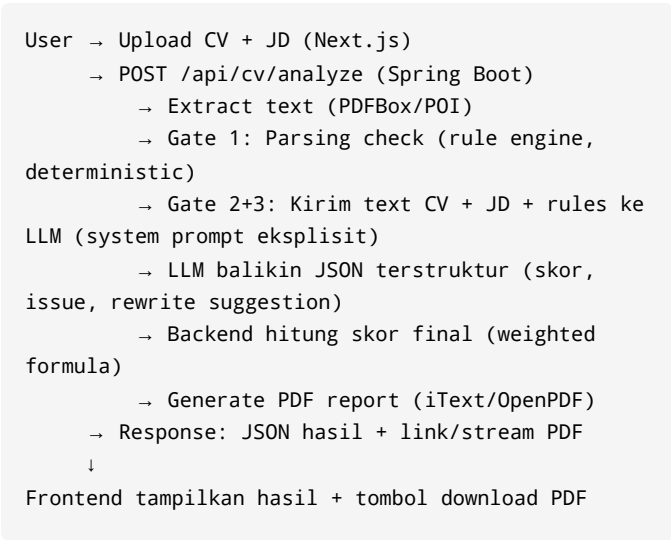
Gate 3 — Scoring & Kelengkapan

- Skor total = weighted dari 3 komponen (bisa disesuaikan, contoh awal: Parsing 40% + Keyword Match 40% + Kelengkapan 20%)
- Kelengkapan: cek years of experience, pendidikan, sertifikasi vs requirement di JD

Layer Tambahan — Rewrite Suggestion (nilai jual utama, di luar ATS asli)

- Untuk tiap bullet yang lemah, AI kasih rewrite pakai formula: **Action verb + keyword + hasil terukur**
- Feedback ditulis human-readable, bukan cuma skor

8. System Architecture / Data Flow



Prinsip penting: rule yang sifatnya biner/pasti (Gate 1) dicek pakai code, bukan diserahkan ke AI — supaya hasilnya konsisten tiap request. AI cuma dipakai untuk hal yang butuh pemahaman semantic (Gate 2, rewrite suggestion).

9. API Endpoints (draft awal)

Method	Endpoint	Deskripsi
POST	/api/cv/analyze	Terima file CV (multipart) + JD

Method	Endpoint	Deskripsi
		(text), return hasil analisa JSON
GET	/api/cv/report/{id}	Download PDF report hasil analisa
POST	/api/auth/login	(Opsional) login
GET	/api/cv/history	(Opsional, perlu login) list riwayat analisa user

10. Data Model (draft awal)

AnalysisResult

- id
- cvFileName
- jdText
- parsingScore, keywordScore, completenessScore, totalScore
- issues: List<Issue> (kategori, deskripsi, severity)
- rewriteSuggestions: List<Suggestion> (bulletOriginal, bulletSuggestion)
- createdAt
- userId (nullable, untuk guest)

Config (rules, disimpan terpisah dari kode)

- `headingWhitelist`
- `fontWhitelist`
- `keywordDensityThreshold`
- `scoreWeights` (parsing/keyword/completeness)

11. Auth & Rate Limiting

- **Guest:** bisa langsung pakai tanpa login, dibatasi rate limit (contoh: 3x analisa/hari) berbasis IP/session pakai Bucket4j — supaya biaya API AI gak membengkak
- **Login (opsional):** untuk simpan history + limit lebih longgar

12. Non-Functional Requirements

- Response time analisa: idealnya < 15 detik (tergantung latency LLM)
- Rule config harus bisa diupdate tanpa redeploy (JSON/DB terpisah)
- System prompt AI harus eksplisit include rule dari riset (bukan andalkan general knowledge model)
- Biaya per analisa harus terkontrol — pakai model murah (Haiku) untuk MVP, monitor token usage

13. Out of Scope (MVP ini)

- Auto-generate file CV baru yang udah direwrite penuh
- RAG / database CV pembandingan (baru dibutuhkan kalau nanti ada fitur "cari contoh CV terbaik dari

database besar")

- Multi-bahasa selain Indonesia/Inggris
- Integrasi langsung ke job board

14. Roadmap Setelah MVP

1. **Auto-rewrite mode** — generate CV baru full dengan template ATS-safe, siap download PDF
2. Compare beberapa versi CV
3. History & dashboard progress user (kalau sudah ada auth)
4. Rule config bisa di-crowd-source/update berkala dari riset ATS terbaru